

Ayudantía 10

Repaso para el Examen

Javier Larré

Pontificia Universidad Católica de Chile

javier.larre@uc.cl

26 de junio de 2026

Ayudantía de hoy

- Ayudantía de hoy 1
- Recordatorio 2
 - Qué queda 3
- I1 4
 - Pregunta 1 5
 - Pregunta 2 10
 - Pregunta 3 14
 - Pregunta 4 16
 - Pregunta 5 19
- Polimorfismo 21
 - Overloading 23
 - Subtype 24
 - Parametric 25
- References 26

Recordatorio

- Recuerden que el examen es $\frac{1}{3}$ de la nota final, mientras que cada interrogación es $\frac{1}{6}$ cada uno.
- El examen será el Martes 7 de julio a las 8:20am.

I1



Pregunta 1

I1

Explica clara y precisamente porqué esta gramática **es ambigua**.

$$\begin{aligned} \langle \text{if_statement} \rangle &::= \text{if (} \langle \text{logic_expr} \rangle \text{) } \langle \text{statement} \rangle \\ &\quad | \text{if (} \langle \text{logic_expr} \rangle \text{) } \langle \text{statement} \rangle \text{ else } \langle \text{statement} \rangle \end{aligned}$$

Pregunta 1

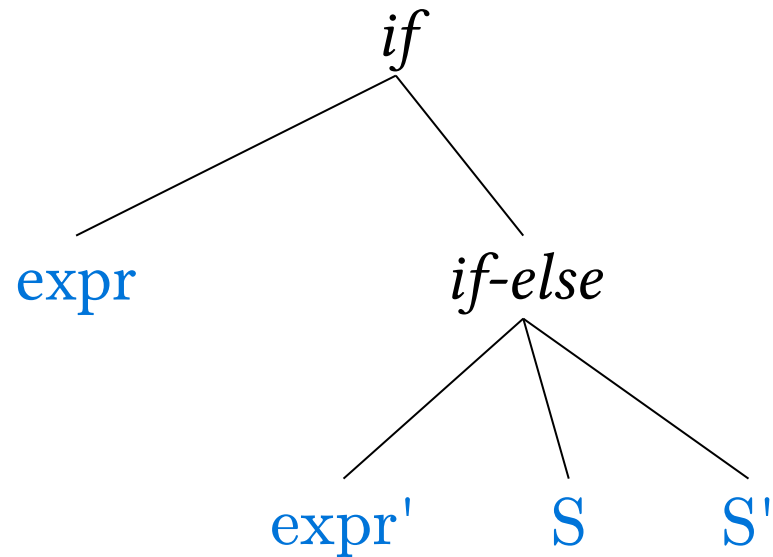
Explica clara y precisamente porqué esta gramática **es ambigua**.

$$\begin{aligned} \langle \text{if_statement} \rangle &::= \text{if (} \langle \text{logic_expr} \rangle \text{) } \langle \text{statement} \rangle \\ &\quad | \text{if (} \langle \text{logic_expr} \rangle \text{) } \langle \text{statement} \rangle \text{ else } \langle \text{statement} \rangle \end{aligned}$$

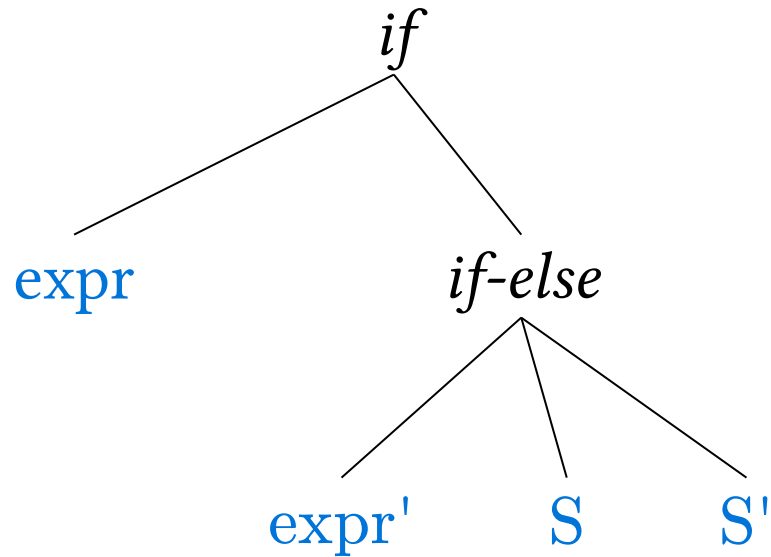
Basta con demostrar que existe un *statement* con dos árboles de derivaciones válidos y distintos.

Usaremos el *statement* **if** (*expr*) **if** (*expr*') *S* **else** (*S*').

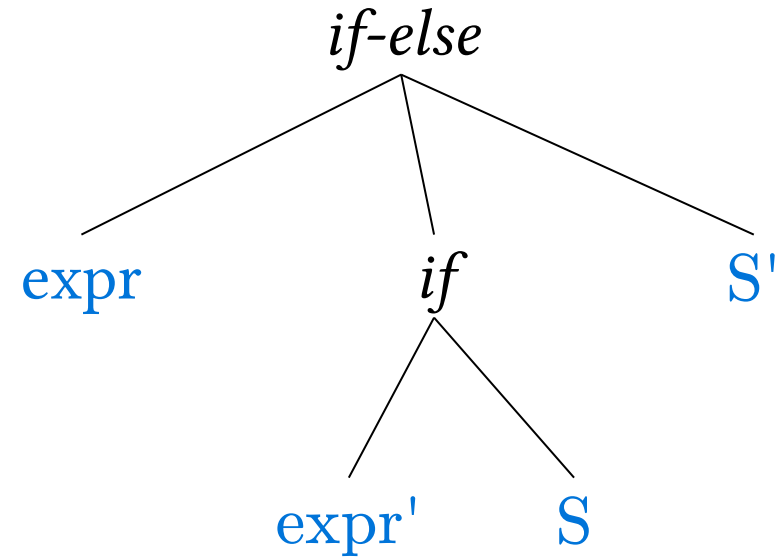
(else anidado al segundo if)



(else anidado al segundo if)



(else anidado al primer if)



Explica clara y precisamente que esta gramática no es ambigua:

$$\langle \text{statement} \rangle ::= \langle \text{matched} \rangle \mid \langle \text{unmatched} \rangle$$
$$\begin{aligned} \langle \text{matched} \rangle ::= & \text{if } (\langle \text{logic_expr} \rangle) \langle \text{matched} \rangle \text{ else } \langle \text{matched} \rangle \\ & \mid \text{any non-if statement} \end{aligned}$$
$$\begin{aligned} \langle \text{unmatched} \rangle ::= & \text{if } (\langle \text{logic_expr} \rangle) \langle \text{statement} \rangle \\ & \mid \text{if } (\langle \text{logic_expr} \rangle) \langle \text{matched} \rangle \text{ else } \langle \text{unmatched} \rangle \end{aligned}$$

Pregunta 1

I1

$\langle \text{statement} \rangle \Rightarrow$ particiona el lenguaje en dos casos: $\langle \text{matched} \rangle$ y $\langle \text{unmatched} \rangle$.

Pregunta 1

I1

$\langle \text{statement} \rangle \Rightarrow$ particiona el lenguaje en dos casos: $\langle \text{matched} \rangle$ y $\langle \text{unmatched} \rangle$.

$\langle \text{matched} \rangle \Rightarrow$ solo puede derivar sentencias donde todo if tiene su else.

$\langle \text{statement} \rangle \Rightarrow$ particiona el lenguaje en dos casos: $\langle \text{matched} \rangle$ y $\langle \text{unmatched} \rangle$.

$\langle \text{matched} \rangle \Rightarrow$ solo puede derivar sentencias donde todo if tiene su else.

$\langle \text{unmatched} \rangle \Rightarrow$ solo puede derivar sentencias donde algún if no lo tiene.

Pregunta 1

$\langle \text{statement} \rangle \Rightarrow$ particiona el lenguaje en dos casos: $\langle \text{matched} \rangle$ y $\langle \text{unmatched} \rangle$.

$\langle \text{matched} \rangle \Rightarrow$ solo puede derivar sentencias donde todo if tiene su else.

$\langle \text{unmatched} \rangle \Rightarrow$ solo puede derivar sentencias donde algún if no lo tiene.

La regla clave es que en $\langle \text{matched} \rangle$, la rama then debe ser también $\langle \text{matched} \rangle$!

Pregunta 1

Esto impide que un else se asocie a un if exterior, ya que si la rama then contiene un if sin else, esa rama es `<unmatched>` y no puede aparecer dentro de `<matched>`. Por lo tanto, **todo else queda forzosamente asociado al if abierto más cercano**, eliminando la ambigüedad del dangling else.

Pregunta 2

Muestra el estado del stack de registros de activación después de ejecutar `fie(2)`. Debes mostrar los punteros de las cadenas estáticas y dinámicas: los nombres locales `x`, `y` y `n` con sus tipos y valores.

```
A: {  
    int y = 0;  
    B: {  
        int x = 0;  
        void fie (int n) {  
            x = n + 1;  
        }  
    }  
}
```

```
        y = n + 2;  
    }  
    C: {  
        int x = 1;  
        fie(2);  
        write(x);  
    }  
}  
write(y);  
}
```


Frame	Variable	Tipo	Valor
A	y	int	0
B	x, fie	int, fun	0, (puntero a fie)
C	x	int	1
fie	n	int	2

Cadena estática: cada registro apunta al registro del bloque que lo contiene léxicamente. `fie` está definida en `B`, por lo que su cadena estática apunta al registro de `B`. `C` está dentro de `B`, por lo que su cadena estática apunta a `B`. `B` está dentro de `A`, por lo que su cadena estática apunta a `A`.

- `fie` $\rightarrow$ `B` $\rightarrow$ `A`
- `C` $\rightarrow$ `B` $\rightarrow$ `A`

Pregunta 2

Cadena dinámica: cada registro apunta al registro del bloque que lo invocó en tiempo de ejecución. El orden de activación es $A \rightarrow B \rightarrow C \rightarrow \text{fie}$, por lo que cada registro apunta al inmediatamente anterior en esa secuencia.

- $\text{fie} \rightarrow C \rightarrow B \rightarrow A$

Pregunta 3

I1

¿Es la siguiente función **tail recursive**?

```
int foo(int a, int b) {  
    if (a == b)  
        return a;  
    else if (a > b)  
        return foo(a-b, b);  
    else  
        return foo(a, b-a);  
}
```

Pregunta 3

¿Es la siguiente función **tail recursive**?

```
int foo(int a, int b) {  
    if (a == b)  
        return a;  
    else if (a > b)  
        return foo(a-b, b);  
    else  
        return foo(a, b-a);  
}
```

En cada caso, $\text{foo}(a-b, b)$ o $\text{foo}(a, b-a)$, se cumple la condición.

Sí: En una función tail recursive, la llamada recursiva es la **última operación ejecutada**.

Pregunta 3

I1

¿Es la siguiente función **tail recursive**?

```
int foo(int x) {  
    if (x > 100)  
        return x-10;  
    else  
        return foo(foo(x+11));  
}
```

Pregunta 3

¿Es la siguiente función **tail recursive**?

```
int foo(int x) {  
    if (x > 100)  
        return x-10;  
    else  
        return foo(foo(x+11));  
}
```

No: Esta función contiene una llamada recursiva **anidada**. El programa debe calcular el resultado de la llamada “interior” primero y luego ejecutar una

segunda llamada recursiva sobre el resultado anterior antes de terminar. Como la llamada exterior está esperando a que la llamada interior se resuelva, **requiere stack frames activos**.

Pregunta 4

I1

Muestra lo que ocurre al ejecutar el siguiente código en un lenguaje que utiliza *lápidas*:

```
p = malloc();  
q = malloc();  
*p = 123;  
*q = 321;  
q = p;  
free(p);
```


Pregunta 4

Muestra lo que ocurre al ejecutar el siguiente código en un lenguaje que utiliza *lápidas*:

```
p = malloc();  
q = malloc();  
*p = 123;  
*q = 321;  
q = p;  
free(p);
```

Cada objeto en el heap tiene asociada una lápida: una **celda intermedia** que contiene la dirección real del objeto. Todos los

punteros al objeto apuntan a la lápida, no directamente al objeto. Al liberar el objeto con `free(p)`, **la lápida se invalida**. Cualquier intento posterior de desreferenciar un puntero que apunta a esa lápida es detectado como **acceso inválido**.

Pregunta 4

I1

Muestra lo que ocurre al ejecutar el siguiente código en un lenguaje que utiliza *candados y llaves* de valores 54321 y 12345 respectivamente:

```
p = malloc();  
q = malloc();  
*p = 123;  
*q = 3;  
q = p;  
free(p);
```


Pregunta 4

Muestra lo que ocurre al ejecutar el siguiente código en un lenguaje que utiliza *candados y llaves* de valores 54321 y 12345 respectivamente:

```
p = malloc();  
q = malloc();  
*p = 123;  
*q = 3;  
q = p;  
free(p);
```

A cada objeto en el heap se le asigna un entero único (candado) que se almacena junto a él. Cada puntero lleva consigo una copia de

ese entero (llave) que se transfiere cuando se asignan a otros punteros (en este caso, 54321 es transferido a q). **Al desreferenciar un puntero se comprueba que su llave coincida con el candado del objeto; si no coinciden, el acceso es inválido.** Al liberar un objeto, su candado se borra (o se

Pregunta 4

I1

cambia), de modo que futuras comparaciones fallen.

Pregunta 5

I1

¿Cuál es el resultado de `a.foo(1)`, `b.foo(1)`, `c.foo(1)` y `d.foo(1)`, ?

```
abstract class A {  
    int val = 1;  
    int foo (int x);  
}  
abstract class B extends A {  
    int val = 2;  
}  
class C extends B {  
    int n = 0;  
    int foo (int x) { return x  
+ val + n; };  
}
```

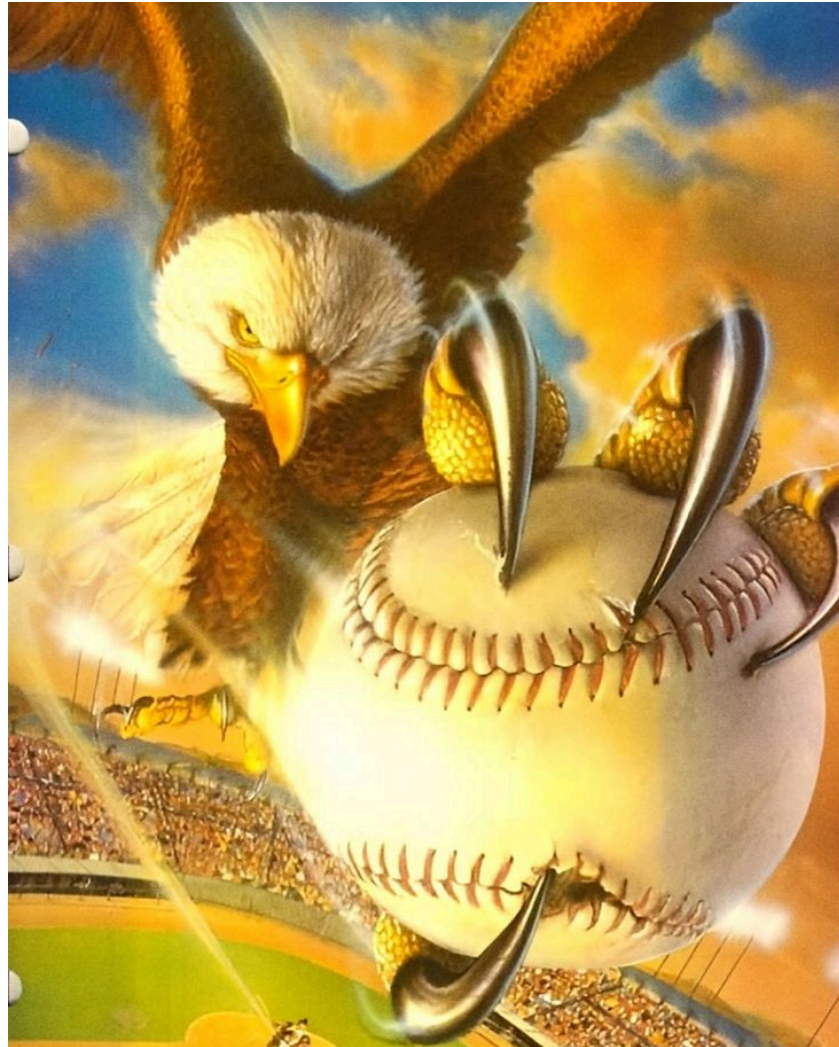
```
class D extends C {  
    int n;  
    D (int v) {n=v;}  
    int foo (int x) { return x  
+ val + n; };  
}  
D d = new D(3);  
A a = d;  
B b = d;  
C c = d;
```

Pregunta 5

I1

Solamente estamos trabajando con el mismo objeto! Por lo que estamos llamando al mismo método (definido en D). El resultado es $1 + 2$ (definido en B) + 3 (entregado en el constructor).

Polimorfismo



A **polymorphic subprogram** takes parameters of different types on different activations. Overloaded subprograms provide a particular kind of polymorphism called ad hoc polymorphism. Overloaded subprograms need not behave similarly.

A **polymorphic subprogram** takes parameters of different types on different activations. Overloaded subprograms provide a particular kind of polymorphism called ad hoc polymorphism. Overloaded subprograms need not behave similarly.

Entonces, ¿cuál es la diferencia entre una función polimórfica y una función con *Overloads*? ¿Puedes dar un ejemplo?

Languages that support **object-oriented programming** usually support **subtype polymorphism**. Subtype polymorphism means that a variable of type T can access any object of type T or any type derived from T.

A more general kind of polymorphism is provided by the methods of Python and Ruby. Recall that variables in these languages do not have types, so formal parameters do not have types. Therefore, a method will work for any type of actual parameter, as long as the operators used on the formal parameters in the method are defined.

Languages that support **object-oriented programming** usually support **subtype polymorphism**. Subtype polymorphism means that a variable of type T can access any object of type T or any type derived from T.

A more general kind of polymorphism is provided by the methods of Python and Ruby. Recall that variables in these languages do not have types, so formal parameters do not have types. Therefore, a method will work for any type of actual parameter, as long as the operators used on the formal parameters in the method are defined.

¿Esto coincide con la definición de *coerción* vista en clases?

Parametric polymorphism is provided by a subprogram that takes generic parameters that are used in type expressions that describe the types of the parameters of the subprogram. Different instantiations of such subprograms can be given different generic parameters, producing subprograms that take different types of parameters. **Parametric definitions of subprograms all behave the same.** Parametrically polymorphic subprograms are often called generic subprograms. C++, Java 5.0+, C# 2005+, and F# provide a kind of compile-time parametric polymorphism. (Sebesta, 2023)

Parametric polymorphism is provided by a subprogram that takes generic parameters that are used in type expressions that describe the types of the parameters of the subprogram. Different instantiations of such subprograms can be given different generic parameters, producing subprograms that take different types of parameters. **Parametric definitions of subprograms all behave the same.** Parametrically polymorphic subprograms are often called generic subprograms. C++, Java 5.0+, C# 2005+, and F# provide a kind of compile-time parametric polymorphism. (Sebesta, 2023)

¿Qué ejemplos hay de este tipo de polimorfismo? ¿Qué casos **no** serían ejemplos de esto?

References

Sebesta, R. (2023). *Concepts of Programming Languages*. Pearson.

The End

Questions? Comments?